

Desenvolvimento Web com .NET

Aula 5 – Listar e Inserir Médicos

Prof. Me. Lucas Teodoro dos Santos

UNIP

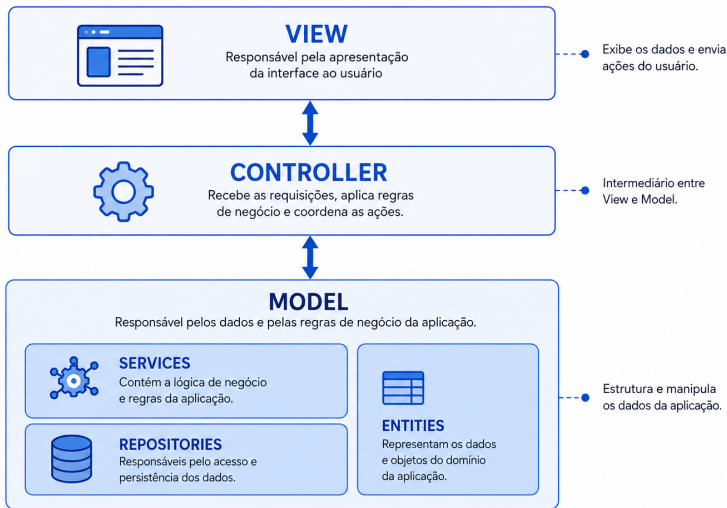
09 de Setembro de 2026

1 Listar Médicos

2 Inserir Médicos

- 1 **AppDbContext**: representa a conexão com o banco de dados e disponibiliza as tabelas da aplicação.
 - `DbSet<Medico>`: representa a tabela de médicos.
- 2 **SeedingService**: insere os dados iniciais no banco, evitando que a tabela de médicos comece vazia.
 - Propriedade `_context`: armazena o contexto utilizado para acessar o banco de dados;
 - Método `Popula()`: verifica se existem médicos cadastrados e insere os dados iniciais quando necessário.
- 3 **MedicoController**: recebe as requisições relacionadas à tela de médicos.
 - Método `Index()`: Página Inicial de listagem de Médicos

Arquitetura MVC



Etapas para criar a página de médicos

Para criar a página inicial do CRUD de médicos, serão realizadas as seguintes etapas:

- ❶ Criar a pasta Services;
- ❷ Criar a classe MedicoService, responsável pelas regras de negócios;
- ❸ Registrar o MedicoService no Program.cs;
- ❹ Criar a MedicoController e receber o MedicoService por Injeção de Dependência;
- ❺ Criar a ação Index() para enviar a lista de médicos para a view;
- ❻ Criar a view Views/Medico/Index.cshtml para apresentar os registros em uma tabela.

Classe MedicoService

A classe MedicoService será responsável por realizar as operações relacionadas aos médicos no banco de dados.

Para isso, ela recebe um objeto AppDbContext por meio da Injeção de Dependência:

```
1 public class MedicoService
2 {
3     private readonly AppDbContext _context;
4
5     public MedicoService(AppDbContext context)
6     {
7         _context = context;
8     }
9 }
```

O objeto _context será utilizado pelos métodos da classe para consultar, inserir, alterar ou remover médicos.

Classe MedicoService - Método Listar()

Para retornar todos os médicos cadastrados, será criado o método Listar():

```
1 public List<Medico> Listar()  
2 {  
3     return _context.Medicos.ToList();  
4 }
```

- List<Medico> indica que o método retorna uma lista de objetos do tipo Medico;
- _context.Medicos acessa os registros da tabela de médicos;
- ToList() executa a consulta e transforma os registros retornados em uma lista;
- Neste projeto, a comunicação com o banco de dados será realizada de forma **síncrona**. A aplicação aguarda a conclusão da consulta antes de continuar a execução do método.

Registrando o MedicoService

Para que o framework consiga criar e fornecer objetos do tipo `MedicoService`, a classe deve ser registrada no `Program.cs`.

O registro deve ser realizado antes da criação da aplicação:

```
1 builder.Services.AddScoped<MedicoService>();
```

O método `AddScoped()` indica que uma instância do serviço será criada dentro de cada escopo de utilização.

Esse ciclo de vida é adequado porque o `MedicoService` depende do `AppDbContext`, que também é utilizado como um serviço *scoped*.

Criando a MedicoController

Deve ser criada uma controller chamada MedicoController na pasta Controllers do projeto.

A MedicoController será responsável por receber as requisições relacionadas aos médicos e enviar os dados para as views.

Para consultar os médicos, a controller recebe o MedicoService por Injeção de Dependência:

```
1 public class MedicoController : Controller
2 {
3     private readonly MedicoService _medicoService;
4
5     public MedicoController(MedicoService medicoService)
6     {
7         _medicoService = medicoService;
8     }
9 }
```

O framework fornece o objeto MedicoService, que será armazenado na variável `_medicoService` para ser utilizado pelas ações da controller.

Ação Index()

A ação Index() solicita a lista de médicos ao serviço e envia o resultado para a view.

```
1 public IActionResult Index()  
2 {  
3     var listaMedicos = _medicoService.Listar();  
4  
5     return View(listaMedicos);  
6 }
```

- `_medicoService.Listar()` consulta os médicos no banco;
- A variável `listaMedicos` armazena os registros retornados;
- `View(listaMedicos)` envia a lista para a view Index.

Criando a View Index

A view responsável por apresentar a lista de médicos deve ser criada na seguinte estrutura:

```
Views/Medico/Index.cshtml
```

O nome da pasta deve corresponder ao nome da controller, removendo o sufixo Controller.

```
MedicoController → Views/Medico
```

Como a ação executada é `Index()`, o arquivo da view deve ser chamado `Index.cshtml`.

Recebendo a lista na View

A ação Index() da controller envia uma lista de médicos para a view:

```
1      return View(listaMedicos);
```

Por isso, a view deve informar o tipo de dado que receberá:

```
1      @model List<Agendamento.Models.Medico>
2
3      @{
4          ViewData["Title"] = "Medicos";
5      }
6
7      <h1>Medicos</h1>
```

A diretiva @model permite utilizar, na view, a lista de objetos Medico enviada pela controller.

Código da Index.cshtml

```
1  @model List<Agendamento.Models.Medico>
2  @{
3      ViewData["Title"] = "Medicos";
4  }
5
6  <h1>Medicos</h1>
7
8  <table class="table table-striped table-hover">
9      <thead>
10         <tr>
11             <th>Id</th>
12             <th>Nome</th>
13             <th>CRM</th>
14             <th>Especialidade</th>
15         </tr>
16     </thead>
17     <tbody>
18         @foreach (var medico in Model)
19         {
20             <tr>
21                 <td>@medico.Id</td>
22                 <td>@medico.Nome</td>
```

Código da Index.cshtml

```
1         <td>@medico.Crm</td>
2         <td>@medico.Especialidade</td>
3     </tr>
4     }
5 </tbody>
6 </table>
```

Criando o link para Médicos

Para acessar a página de listagem, deve ser adicionado um novo item no menu de navegação do arquivo `_Layout.cshtml`.

```
1 <li class="nav-item">
2   <a class="nav-link text-dark" asp-area="" asp-controller="Medico" asp-action="Index">
3     Medicos
4   </a>
</li>
```

- `nav-item` e `nav-link` são classes do Bootstrap utilizadas na criação do menu;
- `asp-controller="Medico"` direciona o link para a `MedicoController`;
- `asp-action="Index"` indica a ação que será executada;
- O texto `Médicos` será apresentado para o usuário no menu.

A funcionalidade de inserção permitirá cadastrar novos médicos no banco de dados por meio de um formulário.

- ❶ Criar um botão na tela de listagem;
- ❷ Criar a ação `Inserir()` com requisição GET;
- ❸ Criar a view com o formulário de cadastro;
- ❹ Criar o método `Inserir()` no `MedicoService`;
- ❺ Criar a ação `Inserir()` com requisição POST;
- ❻ Redirecionar o usuário para a listagem após salvar.

Botão para inserir um novo médico

Na view `Index.cshtml`, será adicionado um link com aparência de botão. Ele direcionará o usuário para a ação `Inserir()` da `MedicoController`.

O botão pode ser colocado após o título e antes da tabela:

```
1 <p>  
2 <a asp-action="Inserir" class="btn btn-success">Inserir</a>  
3 </p>
```

O atributo `asp-action="Inserir"` gera o endereço para a ação `Inserir()` da controller atual.

Requisições GET e POST na inserção

A funcionalidade de inserção utiliza duas ações com o mesmo nome: `Inserir()`.

- A requisição GET ocorre quando o usuário clica no botão para inserir um médico. Ela abre a view com o formulário;
- A requisição POST ocorre quando o usuário clica no botão `Inserir` do formulário. Ela envia os dados preenchidos para a controller;
- A ação POST chama o serviço para salvar o médico e, em seguida, redireciona o usuário para a página `Index`.

Ação Inserir() – GET (MedicoController)

A primeira ação Inserir() será responsável apenas por abrir a tela que contém o formulário de cadastro.

```
1 public IActionResult Inserir()  
2 {  
3     return View();  
4 }
```

Quando o usuário clicar no botão de inserção, será realizada uma requisição GET para a URL:

/Medico/Inserir

A ação retorna a view Views/Medico/Inserir.cshtml.

Criando a view de cadastro

A view do formulário deve ser criada dentro da pasta da controller:

```
Views/Medico/Inserir.cshtml
```

Essa view receberá um objeto do tipo Medico. As propriedades do objeto serão associadas aos campos do formulário.

Os campos utilizados serão:

- Nome;
- CRM;
- Especialidade.

View Inserir.cshtml

A view Inserir.cshtml contém o formulário completo utilizado para receber os dados de um novo médico.

```
1      @model Agendamento.Models.Medico
2
3      <h4>Medico</h4>
4
5      <hr />
6
7      <div class="row">
8          <div class="col-md-4">
9              <form asp-action="Inserir" method="post">
10                  <div class="mb-3">
11                      <label asp-for="Nome" class="form-label"></label>
12                      <input asp-for="Nome" class="form-control" />
13                  </div>
14
15                  <div class="mb-3">
16                      <label asp-for="Crm" class="form-label"></label>
17                      <input asp-for="Crm" class="form-control" />
18                  </div>
```

```
1      <div class="mb-3">
2          <label asp-for="Especialidade" class="form-label"></label>
3          <input asp-for="Especialidade" class="form-control" />
4      </div>
5
6      <div class="mt-3">
7          <input type="submit" value="Inserir" class="btn btn-success" />
8      </div>
9  </form>
10 </div>
11 </div>
```

Os principais elementos utilizados no formulário são:

- `@model Agendamento.Models.Medico`: informa que a view trabalha com um objeto da classe Medico;
- `asp-action="Inserir"`: define que o formulário será enviado para a ação Inserir() da controller;
- `method="post"`: informa que os dados serão enviados para serem processados e gravados;
- `asp-for`: associa os campos do formulário às propriedades do objeto Medico;
- `mb-3`: adiciona espaço abaixo de cada campo;
- `type="submit"`: envia o formulário quando o botão é clicado.

Método Inserir() - MedicoService

O MedicoService será responsável por adicionar o médico ao contexto e salvar a alteração no banco de dados.

```
1 public void Inserir(Medico medico)
2 {
3     _context.Medicos.Add(medico);
4     _context.SaveChanges();
5 }
```

- Add(medico) prepara o novo objeto para ser inserido;
- SaveChanges() efetivamente grava a alteração no banco de dados.

Ação Inserir() – POST (MedicoController)

Após o envio do formulário, a controller recebe um objeto Medico preenchido com os valores digitados pelo usuário.

```
1      [HttpPost]
2      public IActionResult Inserir(Medico medico)
3      {
4          _medicoService.Inserir(medico);
5
6          return RedirectToAction(nameof(Index));
7      }
```

O parâmetro Medico medico é preenchido automaticamente pelo ASP.NET Core a partir dos campos enviados pelo formulário.

Atributo [HttpPost]

O atributo [HttpPost] informa que a ação deve ser executada apenas quando a requisição utilizar o método HTTP POST.

Dessa forma, podem existir duas ações com o mesmo nome:

- Inserir() sem atributo: abre o formulário por meio de uma requisição GET;
- [HttpPost] Inserir(Medico medico): recebe os dados enviados pelo formulário.

A primeira ação apresenta a tela. A segunda processa e salva os dados.

Redirecionamento para a listagem

Depois que o médico é inserido, a ação `Inserir()` redireciona o usuário para a ação `Index()`:

```
return RedirectToAction(nameof(Index));
```

O redirecionamento faz com que a listagem seja carregada novamente, apresentando o médico que acabou de ser cadastrado.

Além disso, evita que o navegador envie novamente o formulário caso o usuário atualize a página.

As primeiras classes criadas são responsáveis por estabelecer o acesso ao banco de dados e disponibilizar os registros iniciais.

- ❶ **AppDbContext**: representa a conexão com o banco de dados e disponibiliza as tabelas da aplicação.
 - `DbSet<Medico>`: representa a tabela de médicos.
- ❷ **SeedingService**: insere os dados iniciais no banco, evitando que a tabela de médicos comece vazia.
 - Propriedade `_context`: armazena o contexto utilizado para acessar o banco de dados;
 - Método `Popula()`: verifica se há médicos cadastrados e insere os dados iniciais quando necessário.

Evolução do Projeto (2/2)

Após preparar o acesso aos dados, foram criadas as classes responsáveis pelas operações de médicos e pelas requisições das telas.

③ **MedicoService**: centraliza as operações relacionadas aos médicos.

- Propriedade `_context`: permite acessar a tabela de médicos;
- Método `Listar()`: retorna todos os médicos cadastrados;
- Método `Inserir(Medico medico)`: adiciona um novo médico e salva a alteração no banco de dados.

④ **MedicoController**: recebe as requisições relacionadas aos médicos e retorna as views.

- Propriedade `_medicoService`: permite utilizar as operações definidas no serviço;
- Método `Index()`: envia a lista de médicos para a view;
- Método `Inserir()`: abre o formulário de cadastro;
- Método `[HttpPost] Inserir(Medico medico)`: recebe os dados do formulário, solicita a inserção e redireciona para a listagem.

Nesta aula foram desenvolvidas novas funcionalidades para o CRUD de médicos na aplicação MVC.

- Foi criada a tela de listagem dos médicos cadastrados;
- Foi desenvolvido o recurso para inserir novos médicos;
- Foram utilizadas views, controllers e serviços para organizar as responsabilidades da aplicação;
- Foram aplicados formulários e requisições HTTP/HTTPS para enviar dados do usuário à aplicação;
- Os dados inseridos passaram a ser salvos no banco de dados.

Obrigado!

Dúvidas?